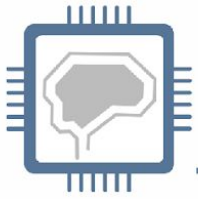


***Inteligência Artificial
e Computacional***

Disciplina: Linguagens de Programação para Inteligência Artificial

Aula 6 - Parte I: Tuplas, Listas e Dicionários

Profa.: Sabrina de A. Silveira

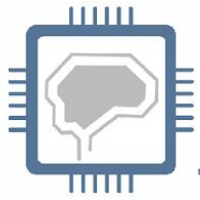


Tuplas

- Uma tupla em Python é uma sequência imutável de valores de qualquer tipo. Para criar uma tupla, lista-se uma sequência de valores separados por vírgulas e, opcionalmente, entre parênteses.
- Cada posição da tupla é indexada por um número inteiro, começando a partir do 0.

```
tupla = ("Inteligência", "Artificial", "e", "Computacional")
```





Tuplas

- Uma tupla em Python é uma sequência imutável de valores de qualquer tipo. Para criar uma tupla, lista-se uma sequência de valores separados por vírgulas e, opcionalmente, entre parênteses.
- Cada posição da tupla é indexada por um número inteiro, começando a partir do 0.

```
tupla = ("Inteligência", "Artificial", "e", "Computacional")
```

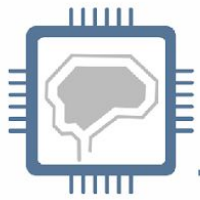
0

1

2

3





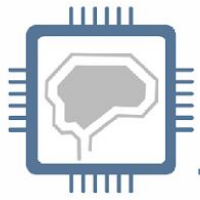
Tuplas

- Uma tupla em Python é uma sequência imutável de valores de qualquer tipo. Para criar uma tupla, lista-se uma sequência de valores separados por vírgulas e, opcionalmente, entre parênteses.
- Cada posição da tupla é indexada por um número inteiro, começando a partir do 0.

```
tupla = ("Inteligência", "Artificial", "e", "Computacional")
```



Conteúdo da
posição de memória



Tuplas

- Uma tupla em Python é uma sequência imutável de valores de qualquer tipo. Para criar uma tupla, lista-se uma sequência de valores separados por vírgulas e, opcionalmente, entre parênteses.
- Cada posição da tupla é indexada por um número inteiro, começando a partir do 0.

```
tupla = ("Inteligência", "Artificial", "e", "Computacional")
```

0

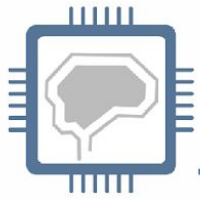
1

2

3

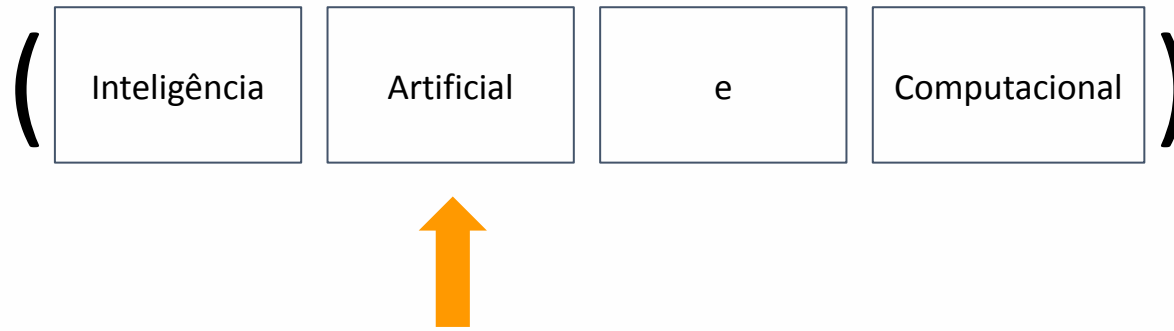


Índice da posição de memória

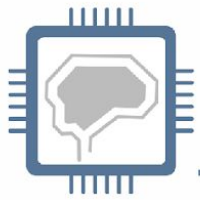


Tuplas

- Para acessar o elemento na quarta posição do tupla usamos a sintaxe tupla[1].
- O conteúdo da posição tupla[1] é a palavra: Artificial.



- tuplas podem ser de 1, 2, 3 ou n dimensões.



Tuplas

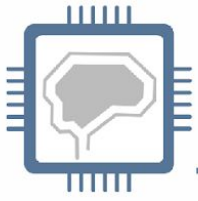
Para trabalharmos com tuplas não precisamos inicializar nenhuma biblioteca.

```
Aluno1 = ("Pedro Henrique", 25, "Pós-Graduando em IA", [60,50,80])
Aluno2 = ("Camila Siqueira", 23, "Pós-Graduando em IA", [40,70,90])

print("O aluno ", Aluno1[0], Aluno1[2], "obteve nota ", Aluno1[3][0], "na primeira prova")
```



- Observe que para criar uma tupla é necessário que os itens estejam entre parêntesis;
- O resultado do print será:
"O aluno Pedro Henrique, Pós-Graduando em IA, obteve nota 60 na primeira prova"
- Nesse exemplo pode ser observado que dentro de uma tupla, podemos inserir outros tipos de dados, como uma lista, por exemplo.



Tuplas - Extraindo informações



```
Aluno2 = ("Camila Siqueira", 23, "Pós-Graduanda em IA", [40,70,90])
```

```
print("O tipo de dados que está sendo utilizado é: ", type(Aluno2))
```

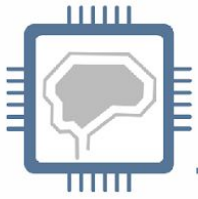
```
print("Tamanho da tupla é: ", len(Aluno2))
```

```
print("Todos os elementos da tupla: ")
```

```
for elemento in Aluno2:
```

```
    print(elemento)
```





Tuplas - Extraindo informações

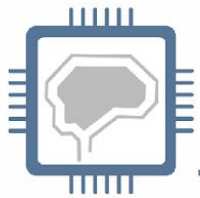
```
Aluno2 = ("Camila Siqueira", 23, "Pós-Graduanda em IA", [40,70,90])

print("O tipo de dados que está sendo utilizado é: ", type(Aluno2))
print("Tamanho da tupla é: ", len(Aluno2))
print("Todos os elementos da tupla: ")
for elemento in Aluno2:
    print(elemento)
```



Os resultados dos prints seriam:

1. O tipo de dados que está sendo utilizado é: <class 'tuple'>
2. O tamanho da tupla é: 4
3. Todos os elementos da tupla:
 - > Camila Siqueira
 - >23
 - >Pós-Graduanda em IA
 - > [40,70,90]



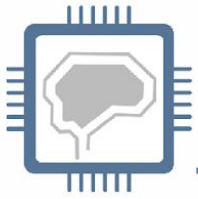
Tuplas são imutáveis!

Case: O aluno foi cadastrado no sistema, mas foi reportado um erro em seu nome e foi solicitado para você corrigir o problema.



```
Aluno3 = ("Gusatvo Almeida", 23, "Pós-Graduando em IA", [40,70,90])
```





Tuplas são imutáveis!

Case: O aluno foi cadastrado no sistema, mas foi reportado um erro em seu nome e foi solicitado para você corrigir o problema.

```
Aluno3 = ("Gusatvo Almeida", 23, "Pós-Graduando em IA", [40,70,90])
```



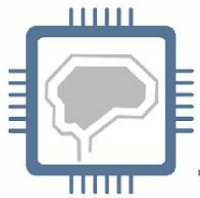
Solução:

```
Aluno3[0] = "Gustavo Almeida"
```



Resultado:

```
TypeError                                Traceback (most recent call last)
<ipython-input-2-ae9421c5a45b> in <module>
      1 Aluno3 = ("Gusatvo Almeida", 23, "Pós-Graduando em IA", [40,70,90])
----> 2 Aluno3[0] = "Gustavo Almeida"
TypeError: 'tuple' object does not support item assignment
```



Tuplas são imutáveis!

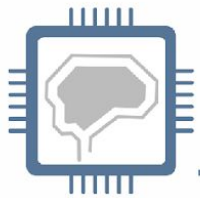
Case: O aluno foi cadastrado no sistema, mas foi reportado um em seu nome



```
Aluno3 = ("Gusatvo Almeida", 23, "Pós-Graduando em IA", [40,70,90])
```



Solução: Se você precisar alterar os elementos em uma tupla, uma alternativa é convertê-la em uma lista, fazer as alterações necessárias e depois converter de volta para uma tupla. Exemplo:



Tuplas são imutáveis!

Case: O aluno foi cadastrado no sistema, mas foi reportado um em seu nome

```
Aluno3 = ("Gusatvo Almeida", 23, "Pós-Graduando em IA", [40,70,90])
```

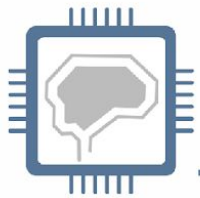


Solução: Se você precisar alterar os elementos em uma tupla, uma alternativa é convertê-la em uma lista, fazer as alterações necessárias e depois converter de volta para uma tupla. Exemplo:

```
aluno3_lista = list(Aluno3) #Convertendo a tupla em lista
aluno3_lista[0] = "Gustavo Almeida" #Fazendo a alteração
Aluno3 = tuple(aluno3_lista) #Convertendo de volta para uma tupla
print(Aluno3)
```



Isso resultará na tupla modificada.



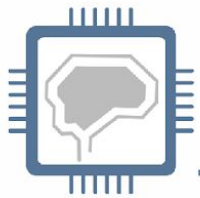
Tuplas - Nota sobre elementos mutáveis

Embora uma tupla em si seja imutável, se a tupla contém elementos mutáveis (como listas), os conteúdos desses elementos podem ser modificados. No exemplo abaixo:



```
Aluno3 = ("Gustavo Almeida", 23, "Pós-Graduando em IA", [40,70,90])
```





Tuplas - Nota sobre elementos mutáveis

Embora uma tupla em si seja imutável, se a tupla contém elementos mutáveis (como listas), os conteúdos desses elementos podem ser modificados. No exemplo abaixo:



```
Aluno3 = ("Gustavo Almeida", 23, "Pós-Graduando em IA", [40,70,90])
```



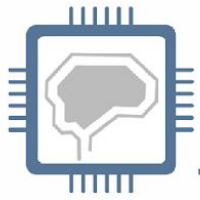
Podemos alterar as notas do aluno fazendo:



```
Aluno3[3][0] = 60
```



Fazendo dessa forma, estamos acessando uma lista que é mutável dentro de um objeto imutável, logo dentro dessa lista podemos executar as operações conhecidas para listas.



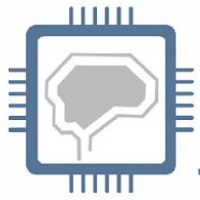
Tuplas - Atribuições de variáveis

Podemos armazenar cada valor da tupla em uma variável, da seguinte forma:



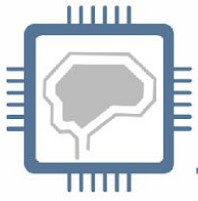
```
Aluno3 = ("Gustavo Almeida", 23, "Pós-Graduando em IA", [40,70,90])
nome, idade, curso, notas = Aluno3
print("Nome do aluno: ", nome)
print("idade: ", idade)
print("Curso: ", curso)
print("Nota P1: ", notas[0])
print("Nota P2: ", notas[1])
print("Nota P3: ", notas[2])
```





Tuplas - Resumo

- Tuplas são uma estrutura de dados em Python que são semelhantes às listas, mas com uma diferença importante: são imutáveis;
- Uma vez criada, uma tupla não pode ser modificada (adicionar, remover ou alterar itens);
- Tuplas são ideias para representar dados que não devem ser alterados, exemplos:
 - Coordenadas geográficas (latitude, longitude);
 - Dimensões de objetos;
 - Pontos em um plano cartesiano.

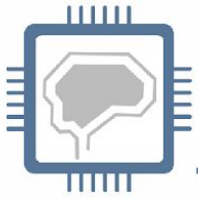


Listas

- Diferente das tuplas, listas podem variar em tamanho e seus elementos podem ser modificados. Listas são mutáveis. Você pode definir uma lista usando os colchetes [].

```
lista = [1,2,3, "casa", None]
```



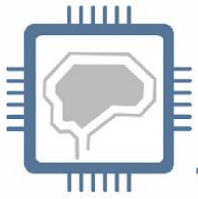


Listas - Utilizando o list()

- As listas em python oferecem algumas facilidades, como por exemplo, transformar um iterador em uma lista de números.

```
gerador = range(10)
lista_gerador = list(gerador) #Saída: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```





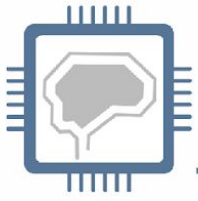
Listas - Utilizando o list()

- As listas em python oferecem algumas facilidades, como por exemplo, transformar um iterador em uma lista de números.

```
gerador = range(10)
lista_gerador = list(gerador) #Saída: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```



- Aqui atribuímos um gerador de números em ordem de 0 a 9 e então usamos a função list e então esse gerador é transformada numa lista organizada de números.

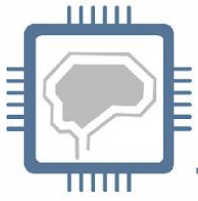


Listas - Método .append

- Adicionar e remover elementos em listas é bem simples, basta utilizarmos o método append.

```
carros = ["Gol", "Monza", "Corsa"]  
carros.append("Clio")  
carros #Saída: ["Gol", "Monza", "Corsa", "Clio"]
```



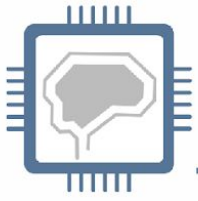


Listas - Método .insert

- Usando o método insert, podemos inserir um elemento em um ponto específico da lista.

```
carros = ["Gol", "Monza", "Corsa", "Clio"]  
carros.insert(1, "Strada")  
carros #Saída: ["Gol", "Strada", "Monza", "Corsa", "Clio"]
```



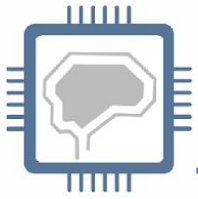


Listas - Método .pop

- Podemos remover um elemento da lista com o método pop, que remove e mostra na tela o elemento removido.

```
carros = ["Gol", "Monza", "Corsa", "Clio"]  
carros.pop(2) #Saída: "Corsa"  
carros #Saída: ["Gol", "Strada", "Corsa", "Clio"]
```



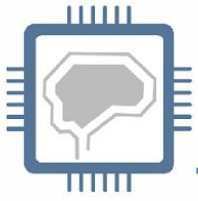


Listas - Método .remove

- Elementos podem ser removidos pelo valor, a função buscará o primeiro valor e então irá removê-lo:

```
carros = ["Gol", "Monza", "Corsa", "Clio"]  
carros.append("Gol")  
carros #Saída: ["Gol", "Strada", "Corsa", "Clio", "Gol"]  
carros.remove("Gol")  
carros #Saída: ["Strada", "Corsa", "Clio", "Gol"]
```



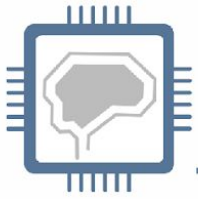


Listas - Busca por valor

- Podemos também verificar se um dado elemento está presente na lista, fazendo:

```
carros = ["Gol", "Monza", "Corsa", "Clio"]  
"Mclaren" in carros #Saída: False
```



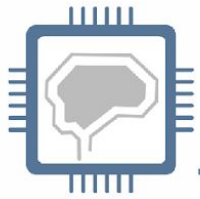


Listas - Busca por valor

- Ou fazer o contrário:

```
carros = ["Gol", "Monza", "Corsa", "Clio"]  
"Mclaren" not in carros #Saída: True
```



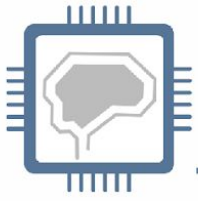


Listas - Concatenação

- Listas também podem ser combinadas, de maneira bem simples. Basta utilizar o operador '+' para concatenar duas ou mais listas

```
valores = [1,2,3,4]
objetos = ['casa', 'carro', 'moto']
lista_concatenada = valores + objetos
lista_concatenada #Saída: [1,2,3,4, 'casa', 'carro', 'moto']
```



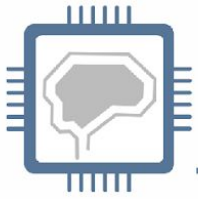


Listas - Ordenação

- Listas podem ser ordenadas sem a necessidade do programador se preocupar em fazer operações e iterações (loops). Em Python basta utilizarmos a função sort.

```
a = [7,2,5,1,3]
a.sort()
a #Saída: [1,2,3,5,7]
```





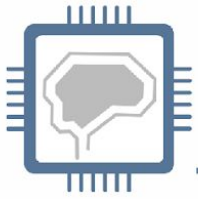
Listas - Ordenação

- A função sort carrega algumas ferramentas que podem ser úteis. Por exemplo, o parâmetro *key*, permite definir uma forma específica de sort, na qual as palavras são organizadas em uma lista com base na quantidade de letras



```
palavras = ["programação", "engenharia", "dados", "análise", "ciência"]  
palavras.sort(key = len)  
palavras #Saída: ["dados", "análise", "ciência", "engenharia", "programação"]
```



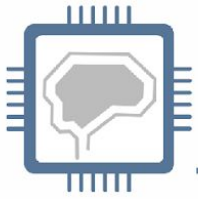


Listas - Fatiamento

- Podemos fatiar listas em python, isto é, através de comandos simples podemos obter subconjuntos de uma lista. Por exemplo:

```
seq = [7,2,3,7,5,6,0,1]  
print(seq[1:5]) #Saída: [2,3,7,5]
```



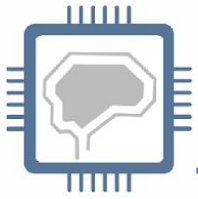


Listas - Fatiamento

- Podemos também utilizar do fatiamento para substituir elementos em uma lista.

```
seq = [7,2,3,7,5,6,0,1]
print(seq[3:5]) #Saída: [7,5]
seq[3:5] = [10,11]
print(seq) #Saída: [7,2,3,10,11,6,0,1]
```





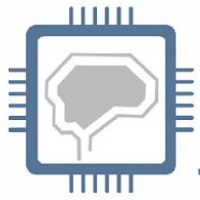
Listas - Fatiamento

- Podemos também utilizar do fatiamento para substituir elementos em uma lista.

```
seq = [7,2,3,7,5,6,0,1]
print(seq[3:5]) #Saída: [7,5]
seq[3:5] = [10,11]
print(seq) #Saída: [7,2,3,10,11,6,0,1]
```



- Aqui podemos observar que, enquanto o índice inicial é incluso, o índice final não é. Podemos dizer que o número de elementos resultantes sempre será **stop - start**.

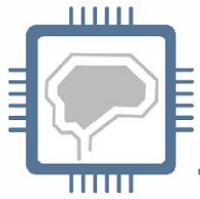


Listas - Fatiamiento

- O start e o stop do fatiamento pode ser omitido também e isto produzirá resultados interessantes.

```
seq = [7,2,3,6,3,6,0,1]
print(seq[:5]) #Saída: [7,2,3,6,3]
print(seq[3:]) #Saída: [6,3,6,0,1]
```



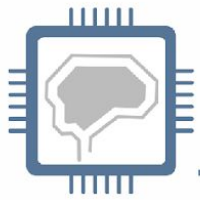


Listas - Fatiamento

- Podemos utilizar também valores negativos no fatiamento.

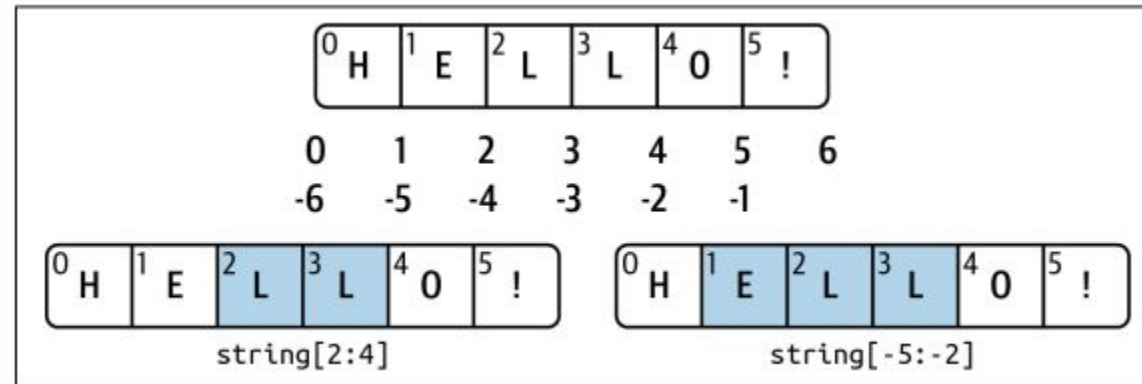
```
seq = [7,2,3,6,3,6,0,1]
print(seq[-4:]) #Saída: [3,6,0,1]
print(seq[-6:-2]) #Saída: [3,6,3,6]
```

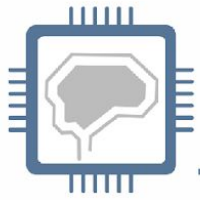




Listas - Fatiamento

- Para facilitar o entendimento do fatiamento de listas a figura abaixo mostra como os índices negativos funcionam





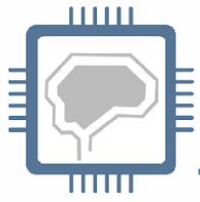
Dicionários

- Dicionários são uma estrutura de dados em Python que armazenam pares de chave-valor.
 - Extremamente úteis quando você precisa associar informações ou quando a busca por valores com base em uma chave é frequente.

```
meu_dicionario = {}      #Criar um dicionário vazio

dados_pessoais = {      #Criar com elementos
    "nome": "João",
    "idade": 30,
    "cidade": "São Paulo",
}
```

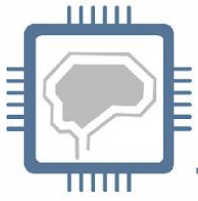




Dicionários - Acessando valores

```
print(dados_pessoais["nome"])    # Saída: João  
print(dados_pessoais["idade"])  # Saída: 30
```



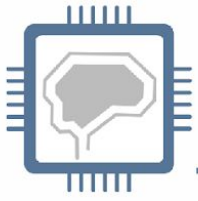


Dicionários - Modificando valores

```
dados_pessoais["idade"] = 31
print(dados_pessoais)
#Saída: {'nome': 'João', 'idade': 31, 'cidade': 'São Paulo'}
```



- Quando utilizamos o comando 'print' e passamos somente a variável dados_pessoais, será impresso todo o dicionário.

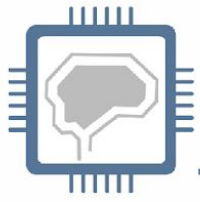


Dicionários - Adicionar novos valores

```
dados_pessoais["email"] = "joao@example.com"  
print(dados_pessoais)  
#Saída: {'nome': 'João', 'idade': 31, 'cidade': 'São Paulo', 'email': 'joao@example.com'}
```



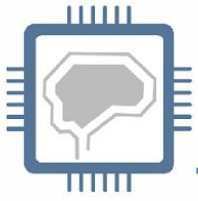
- Para adicionar um elemento no dicionário é bem simples, basta passarmos o nome da chave “email” e o conteúdo dessa chave “joao@example.com”



Dicionários - Removendo chave-Valor

```
del dados_pessoais["cidade"]  
print(dados_pessoais)  
#Saída: {'nome': 'João', 'idade': 31, 'email': 'joao@example.com'}
```





Dicionários - Imprimir todos os elementos



```
for chave, valor in dados_pessoais.items():  
    print(f"{chave}: {valor}")
```

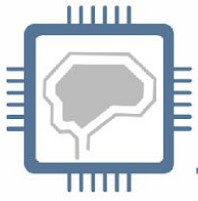
#Saída:

#Nome: João

#idade: 31

#email: joao@example.com





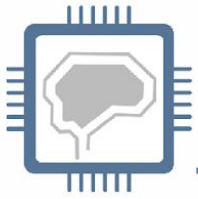
Dicionários - Imprimir todos os elementos

```
dados_pessoais = {      #Criar com elementos
    "nome": "João",
    "idade": 30,
    "cidade": "São Paulo",
}
```



```
dados_pessoais = {      #Criar com elementos
    "nome": "João",
    "idade": 31,
    "email": "joão@example.com",
}
```





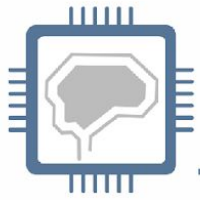
Dicionários - n-dimensões

- Um exemplo mais prático, podemos pensar no cadastro de alunos no sistema da UFV.

```
alunos_pos_ia = {"96855": {"nome": "Carlos", "idade": 28, "cidade": "Viçosa", "notas":  
[70,65,90]},  
                 "100356": {"nome": "Maria", "idade": 30, "cidade": "Belo Horizonte", "notas":  
[70,90,40]},  
                 "106565": {"nome": "Alex", "idade": 24, "cidade": "Ubá", "notas": [70,90,55]}}
```



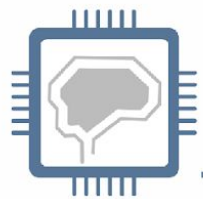
- Agora a chave é o número de matrícula e cada número de matrícula carrega um próprio dicionário com as informações de cada aluno.



Considerações finais

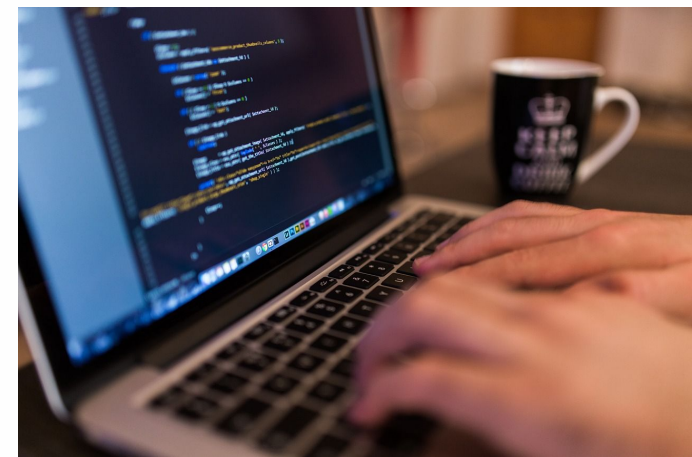
- Todos os códigos utilizados podem ser conferidos no colab. Basta escanear o QRCode.

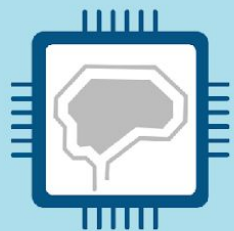




Considerações finais

- Lembre-se: Programação precisa de prática!





Inteligência Artificial e Computacional

Profa.: Sabrina de A. Silveira